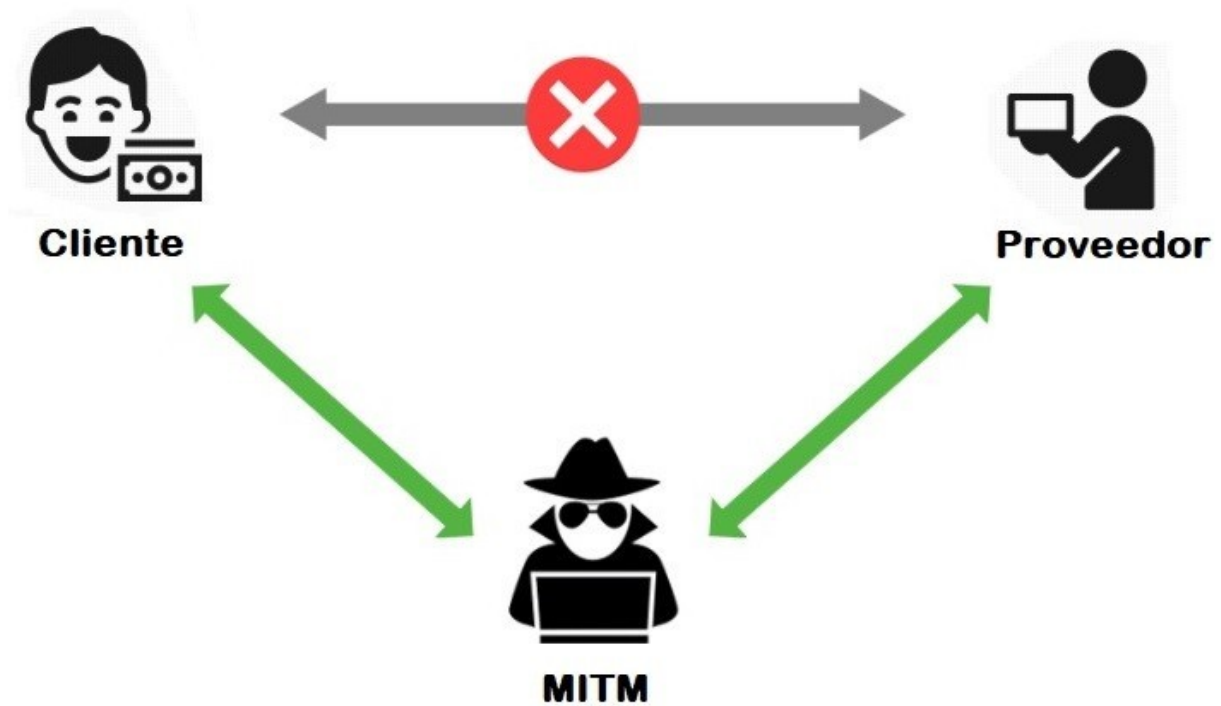


Mitm y suplantación:



TRABAJO REALIZADO POR:

Santiago Pérez Jiménez
2º DE INGENIERÍA DE LA CIBERSEGURIDAD

UNIVERSIDAD EUROPEA
MADRID
03 MAYO 2026

PROFESOR: ALFREDO ROBLEDANO ABASOLO

1. Resumen:

Esta práctica consta de realizar un sistema de monitorización para detectar ataques de envenenamiento de red en Capa 2 (ARP Spoofing) y técnicas de reconocimiento o suplantación mediante DNS (Kaminsky / Fake subdomains).

Consta de dos partes, detección de envenenamiento ARP y la otra parte es suplantación y anomalías DNS.

Índice

1. Resumen:	2
2. Introducción:	5
3. Desarrollo:	6
Parte 1: detección de envenenamiento ARP:	6
Parte 2: suplantación y anomalías DNS:	18
4. Resultados:	27
1. Resultados de la detección de ARP Spoofing	27
2. Resultados de la detección de DNS Snooping	28
5. Conclusiones:	29
6. Bibliografía:	30

Índice de figuras

Figura 1: Docker Compose de ARP.....	6
Figura 2: Iniciar docker compose.....	8
Figura 3: Entrar a la máquina del atacante.....	9
Figura 4: Instalación Bettercap.....	9
Figura 5: Lanzamiento de Bettercap.....	9
Figura 6: Configurando ataque en Bettercap.....	10
Figura 7: Forzar cambio de IP.....	11
Figura 8: Entrada a la herramienta.....	11
Figura 9: Envío de ping.....	12
Figura 10: Herramienta Bettercap.....	12
Figura 11: Generar tráfico.....	13
Figura 12: Muestra de tabla.....	13
Figura 13: Instalación Python.....	14
Figura 14: Instalación Scapy.....	14
Figura 15: Creación del script.....	14
Figura 16: Código detección ARP.....	15
Figura 17: Ejecución del primer código.....	16
Figura 18: Ejecución del ataque.....	16
Figura 19: Ping en máquina víctima.....	17
Figura 20: Funcionamiento de script.....	17
Figura 21: Docker compose de DNS.....	18
Figura 22: Iniciar docker compose DNS.....	19
Figura 23: Entrando a la máquina.....	19
Figura 24: Instalación de Python en máquina DNS.....	19
Figura 25: Instalación de Scapy en máquina DNS.....	19
Figura 26: Código de detección DNS.....	20
Figura 27: Ejecuto script DNS detección.....	21
Figura 28: Generación de tráfico.....	21
Figura 29: Resultado de detección DNS 1.....	22
Figura 30: Creación de archivo de código ataque DNS.....	22
Figura 31: Código de ataque de DNS.....	23
Figura 32: Ejecución de script de ataque DNS.....	24
Figura 33: Resultado detector DNS 1.....	25
Figura 34: Resultado detector DNS 2.....	25

2. Introducción:

En la práctica de hoy va sobre Mitm y suplantación, para ello se divide en dos partes:

1. Detección de envenenamiento ARP:

El ARP Spoofing permite a un atacante redirigir el tráfico de una red local. Su detección es importante para evitar los ataques de Mitm.

En esta parte se diseñará una estrategia para su dicha detección mediante la monitorización y el análisis en las respuestas del protocolo ARP.

2. Suplantación y anomalías DNS:

El ataque de Kaminsky y el “DNS Snooping” aprovechan la resolución de subdominios inexistentes para comprometer la integridad de la caché o mapear la infraestructura interna.

3. Desarrollo:

Parte 1: detección de envenenamiento ARP:

El ARP Spoofing permite a un atacante redirigir el tráfico de una red local. Su detección es importante para evitar los ataques de Mitm.

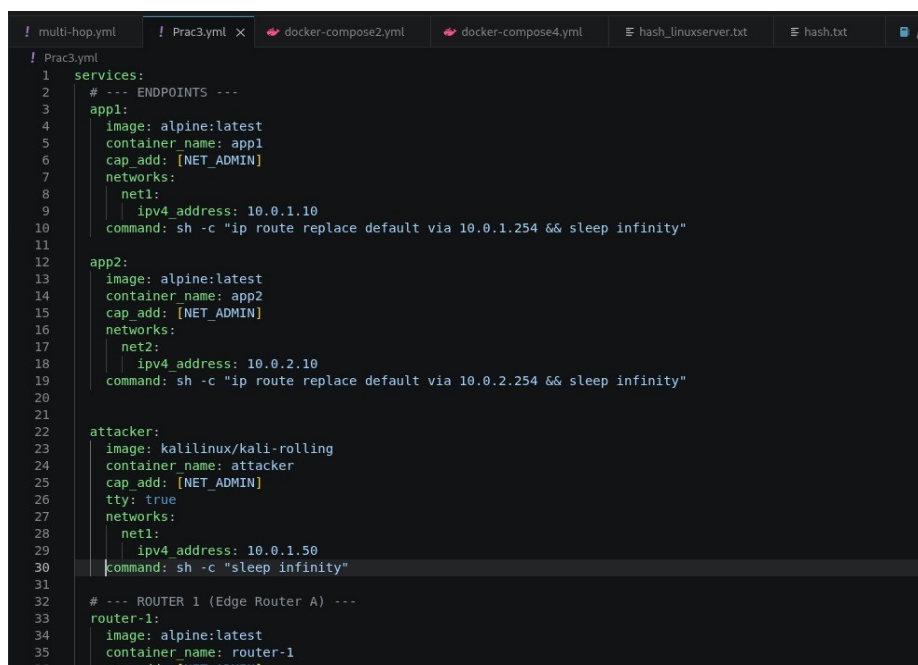
En esta parte se diseñará una estrategia para su dicha detección mediante la monitorización y el análisis en las respuestas del protocolo ARP.

2. (Envenenamiento De Caché ARP | ESET Glossary, n.d.)

Las tareas a realizar nos indican lo siguiente:

- Definir un docker-compose que incluya un nodo víctima, un nodo router y un nodo servidor web en una red externa a la víctima.
- Investigar la herramienta “bettercap” para iniciar el proceso de envenenamiento de las tablas ARP de la víctima y el router.
- Definir una función llamada “alert_arp spoof” en scapy que monitorice e imprima en pantalla las anomalías en las respuestas ARP que permita detectar a un intruso que este realizando el envenenamiento.

Primero para ello voy a generar un docker compose:



```
1 services:
2   # --- ENDPOINTS ---
3   app1:
4     image: alpine:latest
5     container_name: app1
6     cap_add: [NET_ADMIN]
7     networks:
8       net1:
9         ipv4_address: 10.0.1.10
10    command: sh -c "ip route replace default via 10.0.1.254 && sleep infinity"
11
12   app2:
13     image: alpine:latest
14     container_name: app2
15     cap_add: [NET_ADMIN]
16     networks:
17       net2:
18         ipv4_address: 10.0.2.10
19    command: sh -c "ip route replace default via 10.0.2.254 && sleep infinity"
20
21   attacker:
22     image: kalilinux/kali-rolling
23     container_name: attacker
24     cap_add: [NET_ADMIN]
25     tty: true
26     networks:
27       net1:
28         ipv4_address: 10.0.1.50
29    command: sh -c "sleep infinity"
30
31   # --- ROUTER 1 (Edge Router A) ---
32   router-1:
33     image: alpine:latest
34     container_name: router-1
35     cap_add: [NET_ADMIN]
```

Figura 1: Docker Compose de ARP

El docker compose se compone de dos hosts que son “app1” y “app2”, tiene 3 routers, un atacante y redes intermedias “net1”, “net12”, “net23” y “net2”

Me piden que investigue sobre la herramienta “Bettercap”.

Bettercap es una herramienta de seguridad de redes que se utiliza para el análisis, la monitorización y para pruebas de ataques en entornos controlados.

Está pensado para hacer:

- Auditoría de redes.
- Pruebas de penetración (pentesting).
- Análisis del tráfico en tiempo real
- Simulación de ataques en entornos de laboratorio.

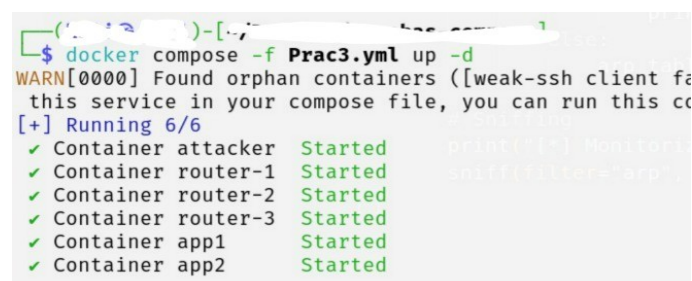
Bettercap permite interactuar con el tráfico de una red de forma activa, es capaz de detectar automáticamente IP's activas, direcciones MAC, hosts y servicios.

Permite ataques de suplantación, como ARP spoofing, DNS spoofing o interceptación de tráfico.

En este caso lo he utilizado para simular un ataque de un atacante en la red, para generar tráfico malicioso (ARP spoofing) y para comprobar que el sistema de detección lo identifica.

1. (Bettercap: Framework Para Hacer MitM Portable Y Modular, n.d.)

Ahora voy a utilizarla para ello inicio el docker compose que he mostrado anteriormente:



```
$ docker compose -f Prac3.yml up -d
WARN[0000] Found orphan containers ([weak-ssh client fa
this service in your compose file, you can run this co
[+] Running 6/6
 ✓ Container attacker Started
 ✓ Container router-1 Started
 ✓ Container router-2 Started
 ✓ Container router-3 Started
 ✓ Container app1 Started
 ✓ Container app2 Started
```

Figura 2: Iniciar docker compose

Entro en la máquina del atacante:

```
$ docker exec -it attacker sh
/ #
/ #
```

Figura 3: Entrar a la máquina del atacante

Ahora dentro de la máquina instalaré la herramienta de Bettercap:

```
# apt install bettercap -y
Installing:
  bettercap

Installing dependencies:
  bettercap-caplets  dmsetup  libcap2-bin  libi
  ca-certificates    iproute2  libcryptsetup12  libi
  dbus               iptables  libdbus-1-3     libj
  dbus-bin          iw        libdevmapper1.02.1  libj
  dbus-daemon       libapparmor1  libedit2        libk
  dbus-session-bus-common  libbpf1     libelf1t64      libn
  dbus-system-bus-common  libcap2     libexpat1       libr
```

Figura 4: Instalación Bettercap

Ahora vamos a iniciar Bettercap para ello iniciamos el “ataque”:

```
# bettercap -iface eth0
bettercap v2.41.5 (built for linux amd64 with go1.24.9) [type 'help' for a list of commands]
0.0.1.0/24 > 10.0.1.50 » [00:24:33] [sys.log] [inf] gateway monitor started ...
0.0.1.0/24 > 10.0.1.50 »
```

Figura 5: Lanzamiento de Bettercap

Con ese comando lo que hacemos es iniciar Bettercap en la interfaz de red que sé específica, en este caso es “eth0”.

Ahora lo que voy a hacer es configurar el ataque:

```
10.0.1.0/24 > 10.0.1.50 » net.[00:27:27] [sys.log] [err] error getting ipv4 gateway: Could not find mac for 10.0.1.1
10.0.1.0/24 > 10.0.1.50 » net.probe on
10.0.1.0/24 > 10.0.1.50 » [00:27:30] [sys.log] [inf] zerogod service discovery started
10.0.1.0/24 > 10.0.1.50 » [00:27:30] [sys.log] [inf] net.probe probing 256 addresses on 10.0.1.0/24
10.0.1.0/24 > 10.0.1.50 » [00:27:32] [sys.log] [err] error getting ipv4 gateway: Could not find mac for 10.0.1.1
10.0.1.0/24 > 10.0.1.50 » net.show
```

IP ▲	MAC	Name	Vendor	Sent	Recvd	Seen
10.0.1.50	7a:8c:35:4a:6b:a1	eth0		0 B	0 B	00:24:33
10.0.1.1	92:dd:4a:65:06:9b	gateway		0 B	0 B	00:24:33
10.0.1.10	2a:f7:83:57:20:f0	appl.pruebas-compose_net1.		2.3 kB	1.7 kB	00:27:30
10.0.1.254	c6:2a:68:93:87:8c	router-1.pruebas-compose_net1.		2.3 kB	1.7 kB	00:27:33

↑ 261 kB / ↓ 623 kB / 14368 pkts

```
10.0.1.0/24 > 10.0.1.50 » set[00:27:37] [sys.log] [err] error getting ipv4 gateway: Could not find mac for 10.0.1.1
10.0.1.0/24 > 10.0.1.50 » set[00:27:42] [sys.log] [err] error getting ipv4 gateway: Could not find mac for 10.0.1.1
10.0.1.0/24 > 10.0.1.50 » set arp.spoof.target[00:27:47] [sys.log] [err] error getting ipv4 gateway: Could not find mac for 10.0.1.1
10.0.1.0/24 > 10.0.1.50 » set arp.spoof.targets 10.0.1[00:27:52] [sys.log] [err] error getting ipv4 gateway: Could not find mac for 10.0.1.1
10.0.1.0/24 > 10.0.1.50 » set arp.spoof.targets 10.0.1.10
10.0.1.0/24 > 10.0.1.50 » arp.spo[00:27:57] [sys.log] [err] error getting ipv4 gateway: Could not find mac for 10.0.1.1
10.0.1.0/24 > 10.0.1.50 » arp.spoof on
10.0.1.0/24 > 10.0.1.50 » [00:27:59] [sys.log] [err] module arp.spoof is already running
10.0.1.0/24 > 10.0.1.50 » [00:28:02] [sys.log] [err] error getting ipv4 gateway: Could not find mac for 10.0.1.1
10.0.1.0/24 > 10.0.1.50 » set net.ga[00:28:07] [sys.log] [err] error getting ipv4 gateway: Could not find mac for 10.0.1.1
10.0.1.0/24 > 10.0.1.50 » set net.gateway 10.0.1[00:28:12] [sys.log] [err] error getting ipv4 gateway: Could not find mac for 10.0.1.1
10.0.1.0/24 > 10.0.1.50 » set net.gawat 10.0.1.254
10.0.1.0/24 > 10.0.1.50 » set net.gateway 10.0.1.254[00:28:17] [sys.log] [err] error getting ipv4 gateway: Could not find mac for 10.0.1.1
10.0.1.0/24 > 10.0.1.50 » arp.spoof on
10.0.1.0/24 > 10.0.1.50 » [00:28:20] [sys.log] [err] module arp.spoof is already running
10.0.1.0/24 > 10.0.1.50 » [00:28:22] [sys.log] [err] error getting ipv4 gateway: Could not find mac for 10.0.1.1
10.0.1.0/24 > 10.0.1.50 » [00:28:27] [sys.log] [err] error getting ipv4 gateway: Could not find mac for 10.0.1.1
10.0.1.0/24 > 10.0.1.50 » [00:28:32] [sys.log] [err] error getting ipv4 gateway: Could not find mac for 10.0.1.1
10.0.1.0/24 > 10.0.1.50 » [00:28:37] [sys.log] [err] error getting ipv4 gateway: Could not find mac for 10.0.1.1
10.0.1.0/24 > 10.0.1.50 » [00:28:42] [sys.log] [err] error getting ipv4 gateway: Could not find mac for 10.0.1.1
10.0.1.0/24 > 10.0.1.50 » set net.gawat [00:28:47] [sys.log] [err] error getting ipv4 gateway: Could not find mac for 10.0.1.1
10.0.1.0/24 > 10.0.1.50 » set net.gateway 10.0.1.254
10.0.1.0/24 > 10.0.1.50 » [00:28:52] [sys.log] [err] error getting ipv4 gateway: Could not find mac for 10.0.1.1
10.0.1.0/24 > 10.0.1.50 » [00:28:57] [sys.log] [err] error getting ipv4 gateway: Could not find mac for 10.0.1.1
10.0.1.0/24 > 10.0.1.50 » set net.gateway 10.0.1.254[00:29:02] [sys.log] [err] error getting ipv4 gateway: Could not find mac for 10.0.1.1
10.0.1.0/24 > 10.0.1.50 » arp.spoof on
```

Figura 6: Configurando ataque en Bettercap

Ahora lo que estoy haciendo es configurar el ataque, para ello se usan 5 comandos, “arp.spoof on”, “net.probe on”, “net.show”, “set arp.spoof.targets” y “set net.gateway”

Por orden, el primero lo que hace es activar el ataque de envenenamiento de ARP, el segundo lo que hace es mandar paquete de sondeo de forma constante a cada dirección IP, el tercero es para detectar en pantalla todos los dispositivos detectados en la red local hasta ese momento, el cuarto es para especificar los “targets” que queremos que Bettercap se centre, que sea más agresivo con lo marcado y el último de todos, sirve para cambiar manualmente la dirección IP de la puerta de enlace.

El principal problema que estoy encontrando es que no encuentra la MAC que marca y entonces puse otra con el “net.gateway”, pero no lo detecta, entonces me obligan a hacerlo fuera de la máquina.

Desde fuera lo que haré es forzar a que la sea la “10.0.1.254”:

```
# ip route
default via 10.0.1.1 dev eth0
10.0.1.0/24 dev eth0 proto kernel scope link src 10.0.1.50
# ip route del default
# ip route add default via 10.0.1.24^H
Error: any valid address is expected rather than "10.0.1.2".
# ^[[A : not found
# : 22:
# ip route add default via 10.0.1.254
# ip route
default via 10.0.1.254 dev eth0
10.0.1.0/24 dev eth0 proto kernel scope link src 10.0.1.50
#
```

Figura 7: Forzar cambio de IP

Como se aprecia está en otra IP, entonces le fuerzo a que sea la IP que yo quiero, entonces volvemos a entrar en la herramienta Bettercap, para ver si lo he solucionado:

```
# bettercap -iface eth0
bettercap v2.41.5 (built for linux amd64 with go1.24.9) [type 'help' for a list of commands]

10.0.1.0/24 > 10.0.1.50 » [00:33:18] [sys.log] [inf] gateway monitor started ...
10.0.1.0/24 > 10.0.1.50 » net.probe on
10.0.1.0/24 > 10.0.1.50 » [00:33:29] [sys.log] [inf] net.probe starting net.recon as a requirement for net.probe
10.0.1.0/24 > 10.0.1.50 » [00:33:29] [endpoint.new] endpoint 10.0.1.10 detected as 2a:f7:83:57:20:f0.
10.0.1.0/24 > 10.0.1.50 » [00:33:29] [sys.log] [inf] net.probe probing 256 addresses on 10.0.1.0/24
10.0.1.0/24 > 10.0.1.50 » [00:33:29] [sys.log] [inf] zerogod service discovery started
10.0.1.0/24 > 10.0.1.50 » [00:33:29] [endpoint.new] endpoint 10.0.1.1 detected as 92:dd:4a:65:06:9b.
10.0.1.0/24 > 10.0.1.50 » set arp.spoof.targets 10.0.1.10
10.0.1.0/24 > 10.0.1.50 » arp.spo[00:33:48] [sys.log] [err] error getting ipv4 gateway: Could not find mac for 10.0.1.254
10.0.1.0/24 > 10.0.1.50 » arp.spoof on
10.0.1.0/24 > 10.0.1.50 » [00:33:52] [sys.log] [inf] arp.spoof arp spoofer started, probing 1 targets.
10.0.1.0/24 > 10.0.1.50 » [00:33:53] [sys.log] [err] error getting ipv4 gateway: Could not find mac for 10.0.1.254
10.0.1.0/24 > 10.0.1.50 » [00:33:58] [sys.log] [err] error getting ipv4 gateway: Could not find mac for 10.0.1.254
10.0.1.0/24 > 10.0.1.50 » [00:34:03] [sys.log] [err] error getting ipv4 gateway: Could not find mac for 10.0.1.254
10.0.1.0/24 > 10.0.1.50 » [00:34:08] [sys.log] [err] error getting ipv4 gateway: Could not find mac for 10.0.1.254
10.0.1.0/24 > 10.0.1.50 » [00:34:13] [sys.log] [err] error getting ipv4 gateway: Could not find mac for 10.0.1.254
10.0.1.0/24 > 10.0.1.50 » [00:34:18] [sys.log] [err] error getting ipv4 gateway: Could not find mac for 10.0.1.254
10.0.1.0/24 > 10.0.1.50 » [00:34:23] [sys.log] [err] error getting ipv4 gateway: Could not find mac for 10.0.1.254
```

Figura 8: Entrada a la herramienta

Como se aprecia en la imagen sigue sin funcionar entonces lo que he pensado es entrar en la terminal de la otra, en la de “app1” y forzarlo con un ping.

Entro en la máquina y le envío un ping a la IP de “10.0.1.254”:

```
/ # ping 10.0.1.254
PING 10.0.1.254 (10.0.1.254): 56 data bytes
64 bytes from 10.0.1.254: seq=0 ttl=64 time=0.240 ms
64 bytes from 10.0.1.254: seq=1 ttl=64 time=0.083 ms
64 bytes from 10.0.1.254: seq=2 ttl=64 time=0.158 ms
64 bytes from 10.0.1.254: seq=3 ttl=64 time=0.163 ms
64 bytes from 10.0.1.254: seq=4 ttl=64 time=0.185 ms
^C
— 10.0.1.254 ping statistics —
5 packets transmitted, 5 packets received, 0% packet loss
```

Figura 9: Envío de ping

Ahora que hice este paso pruebo de nuevo:

```
# bettercap -iface eth0 -eval "set arp.spoof.targets 10.0.1.10; arp.spoof on; net.probe on"
bettercap v2.41.5 (built for linux amd64 with go1.24.9) [type 'help' for a list of commands]

10.0.1.0/24 > 10.0.1.50 » [00:41:33] [sys.log] [war] Could not find mac for 10.0.1.254
10.0.1.0/24 > 10.0.1.50 » [00:41:33] [sys.log] [inf] arp.spoof starting net.recon as a requirement for arp.spoof
10.0.1.0/24 > 10.0.1.50 » [00:41:33] [sys.log] [inf] arp.spoof arp spoofer started, probing 1 targets.
10.0.1.0/24 > 10.0.1.50 » [00:41:33] [endpoint.new] endpoint 10.0.1.10 detected as 2a:f7:83:57:20:f0.
10.0.1.0/24 > 10.0.1.50 » [00:41:33] [endpoint.new] endpoint 10.0.1.1 detected as 92:dd:4a:65:06:9b.
10.0.1.0/24 > 10.0.1.50 » [00:41:33] [sys.log] [inf] zerogod service discovery started
10.0.1.0/24 > 10.0.1.50 » [00:41:33] [sys.log] [inf] net.probe probing 256 addresses on 10.0.1.0/24
```

Figura 10: Herramienta Bettercap

Al entrar de nuevo parece que funciona, la única diferencia que hice es configurarlo desde fuera para tardar menos, pero he puesto todos los comandos mencionados al principio y he especificado la IP a la que quiero atacar y por fin reconoce la gateway.

Entonces reconozco la red, hace un escáner de la red, encuentra los hosts activos, resuelve el IP → MAC y hace un “network discovery”.

Bettercap automáticamente amplía el contexto, lo que hace es detectar la gateway, a los vecinos e intenta construir la tabla ARP.

Ahora lo que me toca es comprobar de que se está efectuando de forma correcta el ataque, para ello en la “app1” voy a generar tráfico:

```
# ping 10.0.1.254
PING 10.0.1.254 (10.0.1.254): 56 data bytes
64 bytes from 10.0.1.254: seq=0 ttl=64 time=0.210 ms
64 bytes from 10.0.1.254: seq=1 ttl=64 time=0.149 ms
64 bytes from 10.0.1.254: seq=2 ttl=64 time=0.168 ms
64 bytes from 10.0.1.254: seq=3 ttl=64 time=0.200 ms
64 bytes from 10.0.1.254: seq=4 ttl=64 time=0.198 ms
64 bytes from 10.0.1.254: seq=5 ttl=64 time=0.171 ms
64 bytes from 10.0.1.254: seq=6 ttl=64 time=0.094 ms
64 bytes from 10.0.1.254: seq=7 ttl=64 time=0.076 ms
64 bytes from 10.0.1.254: seq=8 ttl=64 time=0.094 ms
64 bytes from 10.0.1.254: seq=9 ttl=64 time=0.127 ms
64 bytes from 10.0.1.254: seq=10 ttl=64 time=0.140 ms
64 bytes from 10.0.1.254: seq=11 ttl=64 time=0.138 ms
64 bytes from 10.0.1.254: seq=12 ttl=64 time=0.131 ms
^C
--- 10.0.1.254 ping statistics ---
13 packets transmitted, 13 packets received, 0% packet loss
round-trip min/avg/max = 0.076/0.145/0.210 ms
# ping 10.0.2.10
PING 10.0.2.10 (10.0.2.10): 56 data bytes
64 bytes from 10.0.2.10: seq=0 ttl=61 time=1.014 ms
64 bytes from 10.0.2.10: seq=1 ttl=61 time=0.120 ms
64 bytes from 10.0.2.10: seq=2 ttl=61 time=0.785 ms
64 bytes from 10.0.2.10: seq=3 ttl=61 time=0.178 ms
64 bytes from 10.0.2.10: seq=4 ttl=61 time=0.243 ms
64 bytes from 10.0.2.10: seq=5 ttl=61 time=0.188 ms
64 bytes from 10.0.2.10: seq=6 ttl=61 time=0.188 ms
64 bytes from 10.0.2.10: seq=7 ttl=61 time=0.187 ms
^C
--- 10.0.2.10 ping statistics ---
8 packets transmitted, 8 packets received, 0% packet loss
round-trip min/avg/max = 0.120/0.243/1.014 ms
```

Figura 11: Generar tráfico

Ahora lo que hago es revisar la herramienta de nuevo:

10.0.1.0/24 > 10.0.1.50 » net.show

IP	MAC	Name	Vendor	Sent	Recv	Seen
10.0.1.50	7a:8c:35:4a:6b:a1	eth0		0 B	0 B	00:41:33
10.0.1.1	92:dd:4a:65:06:9b			5.4 kB	4.1 kB	00:47:28
10.0.1.10	2a:f7:83:57:20:f0	app1.pruebas-compose_net1.		5.4 kB	4.1 kB	00:47:28
10.0.1.254	c6:2a:68:93:87:8c	router-1.pruebas-compose_net1.		5.4 kB	4.1 kB	00:47:31

64 bytes from 10.0.2.10: seq=1 ttl=61 time=0.120 ms
64 bytes from 10.0.2.10: seq=2 ttl=61 time=0.785 ms
64 bytes from 10.0.2.10: seq=3 ttl=61 time=0.178 ms
64 bytes from 10.0.2.10: seq=4 ttl=61 time=0.243 ms
64 bytes from 10.0.2.10: seq=5 ttl=61 time=0.188 ms
64 bytes from 10.0.2.10: seq=6 ttl=61 time=0.188 ms

Figura 12: Muestra de tabla

Detecta todo correctamente, detecta los hosts, el router y el gateway de docker que es el 1.1

Detecta que existe un tráfico real, entonces con esta muestra se demuestra que el atacante está dentro y la red responde a sus peticiones.

Ahora lo que me toca es crear el script para poder detectar y monitorizar e imprimir en pantalla “anomalías” en las respuestas de ARP.

Para ello entro en la máquina de “app1” y me instalo las cosas necesarias entre ellas Python y Scapy, en este caso del último el de Scapy lo instalaré en un entorno virtual para poder instalarlo de forma correcta:

```
# apk add python3 py3-pip
1/25) Installing libbz2 (1.0.8-r6)
2/25) Installing libexpat (2.7.5-r0)
3/25) Installing libffi (3.5.2-r0)
4/25) Installing gdbm (1.26-r0)
5/25) Installing xz-libs (5.8.3-r0)
```

Figura 13: Instalación Python

```
/ # python3 -m venv venv
/ # source venv/bin/activate
((venv) ) / # pip install scapy
Collecting scapy
  Downloading scapy-2.7.0-py3-none-any.whl.metadata (5.8 kB)
  Downloading scapy-2.7.0-py3-none-any.whl (2.6 MB)
    2.6/2.6 MB 7.0 MB/s eta 0:00:00
Installing collected packages: scapy
Successfully installed scapy-2.7.0

[notice] A new release of pip is available: 25.0.1 → 26.1.1
[notice] To update, run: pip install --upgrade pip
```

Figura 14: Instalación Scapy

Ahora lo que toca es crear el archivo del script:

```
((venv) ) / # nano detector.py
```

Figura 15: Creación del script

El código sería el siguiente:

```
from scapy.all import *
import time

arp_tabla = {}
ultimo_visto = {}

def alert(pkt):
    if pkt.haslayer(ARP) and pkt[ARP].op == 2:
        ip = pkt[ARP].psrc
        mac = pkt[ARP].hwsrc

        now = time.time()

        # detecta cambios de la MAC
        if ip in arp_tabla and arp_tabla[ip] != mac:
            print("\nARP SPOOFING DETECTADO !!!")
            print(f"IP: {ip}")
            print(f"MAC original: {arp_tabla[ip]}")
            print(f"MAC nueva: {mac}")
            print("-----")

        # detecta muchas repeticiones sospechosas en poco tiempo
        if ip in ultimo_visto and now - ultimo_visto[ip] < 1:
            print(f"Posible ARP Spoofing detectado!! (ARP reply anómola): {ip}")

        arp_tabla[ip] = mac
        ultimo_visto[ip] = now

print("[*] Monitorizando ARP...")
sniff(iface="eth0", filter="arp", prn=alert, store=0)
```

Figura 16: Código detección ARP

Voy a analizar lo que hace el código, para empezar, el script lo que hace escuchar tráfico ARP en la red, guarda las IPs con su MAC y detecta dos anomalías distintas:

- Cuando ocurran un cambio de MAC para una misma IP
- Cuando hay demasiadas respuestas ARP en poco tiempo

Lo muestra mediante una alerta de sospecha.

Las dos primeras líneas del código son librerías necesarias, una es la de scapy para capturar y analizar paquetes en red y el otro es el de “time” que sirve para medir tiempos entre paquetes.

Luego el primer diccionario lo que hago es guardar la IP y la MAC, el segundo diccionario lo que hago es guardar el último momento en el que se vio la IP, me sirve para analizar si hay demasiados ARP en poco tiempo.

Luego tendríamos la función, que se ejecuta con cada paquete. Lo que hago con la primera línea el “if” es separar que solo me analice paquetes ARP y que solo me analice los de respuesta por eso el de “op == 2”.

Las líneas “ip = pkt[ARP].psrc” y “ mac = pkt[ARP].hwsrc”, sirven para recoger la IP y MAC de origen.

La siguiente línea la de “now” me sirve para recoger el tiempo actual en segundos, para poder medir cada cuanto sale el mismo paquete.

El siguiente “if” es para la primera detección, en este caso es para la detección de un cambio de MAC, lo que dice ahí es que si habíamos visto esa IP antes y la MAC actual es distinta a la que he guardado anteriormente hay una inconsistencia y entonces es posible que exista un posible spoofing.

Entonces imprime varias cosas para alertar.

Las siguientes líneas es para la segunda detección la de mucha repetición de tráfico ARP el llamado “flood”

El if nos dice que si ya vimos la IP antes, y han pasado menos de 1 segundo es que hay demasiada actividad ARP, esto se podría ajustar y en vez de ser tan estricto podría poner menos de 4 segundos a lo mejor, si pasa eso nos muestra con un “print” la alerta.

La línea de “arp_tabla[ip] = mac” y la de “ultimo_visto[ip] = now”, lo que hacen es guardar o actualizar la MAC de esa IP y la otra actualiza el tiempo del último ARP visto.

Las dos siguientes líneas es el inicio del sniffer, la primera es para un mensaje de inicio y la otra hace varias cosas:

“sniff(iface=“eth0”, filter=“arp”, prn=alert, store=0)”:

- iface=“eth0”: solo escucha esa interfaz.
- filter=“arp”: solo actúa con paquetes ARP.
- prn=alert: llama a la función “alert()” por cada paquete ARP
- store=0: no guarda los paquetes en memoria, con esto consigo que sea más eficiente.

Tras analizar el código y hacerlo lo que hago es ejecutar el código, hay que tener en cuenta que la máquina de atacante he quitado la herramienta:

```
((venv)) / # python3 detector.py
[*] Monitorizando ARP ...
# arping 10.0.1.254
```

Figura 17: Ejecución del primer código

De momento está en modo escucha no saca nada porque no existe tráfico ARP.

Ahora voy a provocar que haya tráfico ARP y empezar con el ataque:

```
» arp.spoof on
» [01:26:59] [sys.log] [inf] arp.spoof arp spoofer started, probing 2 targets.
» [radon] (ARP reply anomaly): 10.0.1.50
```

Figura 18: Ejecución del ataque

Lo que hago ahora es activar el ataque de envenenamiento de ARP, con la configuración que teníamos antes.

Para asegurarme que hay tráfico ARP también hago ping desde la víctima “app1”:

```
/ # arping 10.0.1.254 3 detector.py
ARPING 10.0.1.254 from 10.0.1.10 eth0
Unicast reply from 10.0.1.254 [c6:2a:68:93:87:8c] 0.009ms 10.0.1.
Unicast reply from 10.0.1.254 [c6:2a:68:93:87:8c] 0.040ms 10.0.1.
Unicast reply from 10.0.1.254 [c6:2a:68:93:87:8c] 0.026ms 10.0.1.
Unicast reply from 10.0.1.254 [c6:2a:68:93:87:8c] 0.042ms 10.0.1.
Unicast reply from 10.0.1.254 [c6:2a:68:93:87:8c] 1.055ms 10.0.1.
■
```

Figura 19: Ping en máquina víctima

Entonces ahora reviso el script para ver si detectó algo:

```
((venv) ) / # python3 detector.py
[*] Monitorizando ARP... 0.1.10 eth0
Posible ARP Spoofing detectado!! (ARP reply anómalo): 10.0.1.50
Posible ARP Spoofing detectado!! (ARP reply anómalo): 10.0.1.50
Posible ARP Spoofing detectado!! (ARP reply anómalo): 10.0.1.50
Posible ARP Spoofing detectado!! (ARP reply anómalo): 10.0.1.50
Posible ARP Spoofing detectado!! (ARP reply anómalo): 10.0.1.50
Posible ARP Spoofing detectado!! (ARP reply anómalo): 10.0.1.50
Posible ARP Spoofing detectado!! (ARP reply anómalo): 10.0.1.50
Posible ARP Spoofing detectado!! (ARP reply anómalo): 10.0.1.50
Posible ARP Spoofing detectado!! (ARP reply anómalo): 10.0.1.254
Posible ARP Spoofing detectado!! (ARP reply anómalo): 10.0.1.254
Posible ARP Spoofing detectado!! (ARP reply anómalo): 10.0.1.254
Posible ARP Spoofing detectado!! (ARP reply anómalo): 10.0.1.254
Posible ARP Spoofing detectado!! (ARP reply anómalo): 10.0.1.254
Posible ARP Spoofing detectado!! (ARP reply anómalo): 10.0.1.254
■
```

Figura 20: Funcionamiento de script

Como se muestra en la imagen la detección fue efectuada de forma correcta.

Para aclarar la imagen la IP 10.0.1.50, es la generada por Bettercap, entonces efectivamente está haciendo el ataque y funcionaría de forma correcta, es el que nos está haciendo el ARP spoofing.

La otra IP, como he mostrado arriba es la que generado aposta para que se viera que funciona el script de forma en el que lo definimos, para poder hacer una prueba de que se viera que funciona correctamente el script.

El problema de Bettercap por lo que me he informado es que no genera un cambio de MAC visible entonces la primera prueba no lo puedo hacer correctamente, se que es fundamental en un ARP spoofing, pero la herramienta no me proporciona dicha cosa.

Parte 2: suplantación y anomalías DNS:

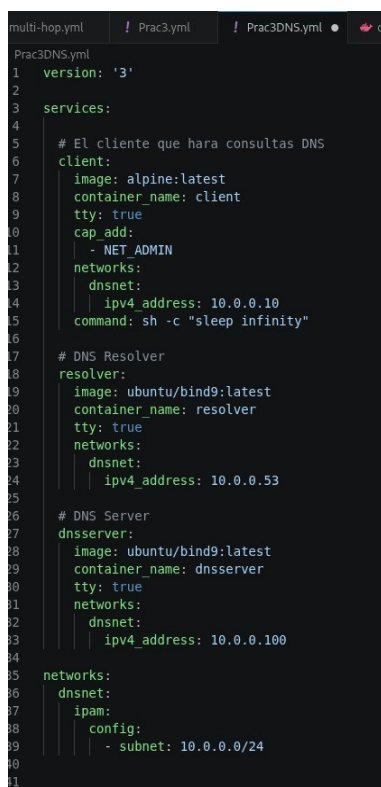
El ataque de Kaminsky y el “DNS Snooping” aprovechan la resolución de subdominios inexistentes para comprometer la integridad de la caché o mapear la infraestructura interna.

3. (Kaushika-Msft, n.d.)

Las tareas a realizar nos indican lo siguiente:

- Definir un docker-compose que incluya un nodo DNS resolver y un servidor DNS.
- Configurar una regla de detección en forma de función en Python mediante la función “alert_dnssnooping” que identifique ráfagas de consultas DNS sospechosas:
 - Firma: detección por volumen (Threshold) de peticiones a subdominios inexistentes.
- Validar el sistema mediante un script en Scapy que genere tráfico hacía un servidor DNS resolver, evidenciando las alertas imprimiendo por pantalla si se cumple la condición.

Primero vamos con el docker-compose:



```
Prac3DNS.yml
1  version: '3'
2
3  services:
4
5      # El cliente que hara consultas DNS
6      client:
7          image: alpine:latest
8          container_name: client
9          tty: true
10         cap_add:
11             - NET_ADMIN
12         networks:
13             dnsnet:
14                 ipv4_address: 10.0.0.10
15         command: sh -c "sleep infinity"
16
17     # DNS Resolver
18     resolver:
19         image: ubuntu/bind9:latest
20         container_name: resolver
21         tty: true
22         networks:
23             dnsnet:
24                 ipv4_address: 10.0.0.53
25
26     # DNS Server
27     dnsserver:
28         image: ubuntu/bind9:latest
29         container_name: dnsserver
30         tty: true
31         networks:
32             dnsnet:
33                 ipv4_address: 10.0.0.100
34
35     networks:
36         dnsnet:
37             ipam:
38                 config:
39                     - subnet: 10.0.0.0/24
40
41
```

Figura 21: Docker compose de DNS

Está compuesto de una red llamada “dnsnet” con rango de IPs de 10.0.0.0/24, son 3 contenedores, un cliente, un DNS resolver y un DNS Server.

Está pensado que desde el cliente hacer consultas, el resolver al no saber la respuesta, viaja por la red y le pregunta al DNS server, este responde y se lo da al resolver que lo guarda y se la entrega al “client”.

Ahora continuamos con el primer script de los dos que nos piden, que es el de detección de DNS, hasta que no haga el otro script que es el de ataque con Scapy no puedo saber si funciona, pero lo primero que voy a hacer es levantar el docker-compose:

```
docker compose -f Prac3DNS.yml up -d
[N[0000] /home/kali/Desktop/pruebas-compose/f
on
Running 5/5
dnsserver Pulled
✓ 196b5efbbc5f Pull complete
✓ 69b182ef14ea Pull complete
1568831-1568831 Pull complete
```

Figura 22: Iniciar docker compose DNS

Y entramos donde queremos que tenga la detección de DNS que será en el DNS resolver:

```
docker exec -it resolver sh
t update
1 http://archive.ubuntu.com/ubuntu pl
2 http://security.ubuntu.com/ubuntu p
3 http://archive.ubuntu.com/ubuntu pl
```

Figura 23: Entrando a la máquina

Instalamos Python y Scapy en esta máquina:

```
# apt install -y python3 python3-pip
Upgrading:
  gpgv
Installing:
  python3 python3-pip
Installing dependencies:
  binutils gcc-14-x86-64-linu
```

Figura 24: Instalación de Python en máquina DNS

```
(venv) # pip install scapy
Collecting scapy
  Downloading scapy-2.7.0-py3-n
  Downloading scapy-2.7.0-py3-non
```

Figura 25: Instalación de Scapy en máquina DNS

Ahora hago el código:

```
from scapy.all import *
import time

dns_counter = {}

THRESHOLD = 5
WINDOW = 10

def alert_dnssnooping(pkt):
    if pkt.haslayer(DNSQR):
        src_ip = pkt[IP].src
        query = pkt[DNSQR].qname.decode()

        now = time.time()

        if src_ip not in dns_counter:
            dns_counter[src_ip] = []

        dns_counter[src_ip].append(now)

        # eliminar consultas antiguas
        dns_counter[src_ip] = [
            t for t in dns_counter[src_ip]
            if now - t < WINDOW
        ]

        print(f"[DNS] {src_ip} -> {query}")

        # deteccion por volumen
        if len(dns_counter[src_ip]) > THRESHOLD:
            print("\n[!!!] Posible DNS Snooping detectado")
            print(f"IP sospechosa: {src_ip}")
            print(f"Consultas DNS en {WINDOW} segundos: {len(dns_counter[src_ip])}")
            print("-----")

print("[*] Monitorizando consultas DNS...")
sniff(filter="udp port 53", prn=alert_dnssnooping, store=0)
```

Figura 26: Código de detección DNS

Antes de nada el script lo que analiza es cuantas consultas se hacen y en cuanto tiempo se hace por IP dicha consulta.

Si en el periodo de 10 segundos se hace más de 5 consultas salta la alerta. Pero bueno ahora especifico por líneas como funciona.

Lo primero las dos primeras líneas son librerías para que funcione con Scapy.

Luego hago un diccionario para guardar una IP y la cantidad de tiempo que hace consulta al DNS.

Luego “THRESHOLD = 5”, es el número máximo de consultas que dejo que tengan de margen, si las supera es una sospecha de posible ataque, si es más de 5 salta la alerta.

Con “WINDOW = 10”, es la ventana de tiempo en segundos, solo cuentan las consultas dentro de los últimos 10 segundos

Pasamos con la función, se ejecuta cada vez que pasa un paquete DNS, ¿cómo?, sencillo con el “if” primero compruebo si el paquete tiene una consulta DNS, el “DNSQR” es “DNS Question Record”, solo analizo consultas DNS reales.

Las dos siguientes líneas “src_ip” y “query” saco la IP que hace la consulta DNS y saco el nombre del dominio consultado. Guardo el tiempo actual en segundos de estos dos datos, con el “now”.

Con el “if” siguiente lo que hacemos es que si es la primera vez que se ve esa IP, crea una lista vacía para guardar los tiempos correspondientes y la siguiente línea “dns_counter” lo que hace es añadir el tiempo actual a la lista de esa IP.

Luego en el “#” lo digo lo que hago es eliminar consultas antiguas solo mantengo consultas que estén dentro de los últimos 10 segundos.

En “print(d”[DNS]....)” lo que hago es imprimir cada consulta de DNS que se haya detectado.

Y ahora en este bloque lo que hago es la detección del ataque, con el “if len” consulto si esa IP hizo más de 5 consultas en 10 segundos, si pasa eso se ejecuta todos esos prints avisando de posible ataque.

Por último, las dos últimas líneas es para capturar el tráfico, la primera es para un mensaje inicial y luego la línea de “sniff(filter=“udp port 53”, prn=alert_dnssnooping, store=0)” es la línea importante lo que significa es lo siguiente:

- sniff(): captura paquetes.
- filter=“udp port 53”: solo captamos esos paquetes.
- prn=alert_dnssnooping: ejecuta la función por cada paquete.
- Store=0: no guardo paquetes en memoria, por eficiencia.

Ahora voy a hacer una prueba simple sin el script de ataque para comprobar que funcione y luego sí que lo hago con el script de ataque, primero ejecuto el script:

```
(venv) # python3 detector_dns.py  
[*] Monitorizando consultas DNS ...
```

Figura 27: Ejecuto script DNS detección

En la máquina de “client” genero tráfico hacía Google:

```
/ # nslookup google.com 10.0.0.53  
Server:      10.0.0.53  
Address:     10.0.0.53:53  
  
Non-authoritative answer:  
Name:   google.com  
Address: 2a00:1450:4003:817::200e  
  
Non-authoritative answer:  
Name:   google.com  
Address: 216.58.205.238
```

Figura 28: Generación de tráfico

Es una consulta sencilla con “nslookup”

Vuelvo al script para revisar que me sacó:

```
# . venv/bin/activate
(venv) # python3 detector_dns.py
[*] Monitorizando consultas DNS...
[DNS] 10.0.0.10 → google.com.
[DNS] 10.0.0.10 → google.com.
[DNS] 10.0.0.53 → google.com.
[DNS] 10.0.0.53 → google.com.
[DNS] 192.48.79.30 → google.com.
[DNS] 192.48.79.30 → google.com.
[DNS] 10.0.0.53 → google.com.
[DNS] 10.0.0.53 → google.com.
[DNS] 216.239.38.10 → google.com.
[DNS] 216.239.38.10 → google.com.
[DNS] 10.0.0.53 → google.com.
[DNS] 192.54.112.30 → google.com.
[DNS] 10.0.0.53 → com.

[!!!] Posible DNS Snooping detectado
IP sospechosa: 10.0.0.53
Consultas DNS en 10 segundos: 6

[DNS] 192.41.162.30 → com.
[DNS] 10.0.0.53 → google.com.

[!!!] Posible DNS Snooping detectado
IP sospechosa: 10.0.0.53
Consultas DNS en 10 segundos: 7

[DNS] 10.0.0.53 → google.com.

[!!!] Posible DNS Snooping detectado
IP sospechosa: 10.0.0.53
```

Figura 29: Resultado de detección DNS 1

Veo que funciona como quería, primero me saca las consultas DNS que hace y luego como ve que ha sido 6 consultas en 10 segundos me salta la alerta marcando la IP sospechosa, el detector identifica patrones de comportamiento anómalos basados en el volumen de consultas DNS que se hace en ese periodo que es 10 segundos. Si nos damos cuenta en realidad en 10 segundos solo dio tiempo a 6 consultas, pero en realidad en más segundos se hizo unas cuantas más, a lo mejor como cambio podría añadir más segundos en ventana o más consultas no sé cuantas consultas es lo normal.

Lo normal, es que esto me dé muchos falsos positivos, por ello que es un laboratorio, este umbral de 5 consultas en 10 segundos si se supera salta alerta, podría variar porque las webs modernas pueden rondar entre las 10 y 50 consultas, por eso es importante remarcar este punto.

Ahora vamos a crear el ataque para ello lo hacemos en “client”:

```
((venv) ) / # nano dns_ataque.py
```

Figura 30: Creación de archivo de código ataque DNS

Código de ataque DNS:

```
from scapy.all import *
import random
import time

DNS_SERVER = "10.0.0.53"

def random_subdomain():
    return ''.join(random.choice("abcdefghijklmnopqrstuvwxyz") for _ in range(10)) + ".test.local"

print("[*] Iniciando ataque de DNS snooping (simulado)...")

for i in range(100):
    domain = random_subdomain()

    pkt = IP(dst=DNS_SERVER)/UDP(dport=53)/DNS(rd=1, qd=DNSQR(qname=domain))

    send(pkt, verbose=0)

    print(f"[DNS QUERY] {domain}")

    time.sleep(0.1)
```

Figura 31: Código de ataque de DNS

Lo primero de todo lo que hace el script es generar subdominios aleatorios, los envío al DNS resolver, creo ráfagas de tráfico DNS y simulo el DNS snooping.

Ahora vamos línea por línea explicando el código.

Las 3 primeras líneas son librerías.

Luego defino la IP del DNS resolver, creo la función y genero subdominios falsos aleatorios, hago que me seleccione letras al azar en una cadena de 10 letras, que las une en una sola palabra y le añado el dominio “.test.local”. Con esto simulo el DNS Snooping.

Genero un mensaje inicial para indicar donde empieza el ataque.

En el bucle le digo que me haga 100 consultas DNS seguidas (a lo mejor para el experimento me he pasado)

Con “domain =” genero un subdominio nuevo en cada iteración.

Construyo el paquete DNS “pkt = IP(dst=DNS_SERVER)/UDP(dport=53)/DNS(rd=1, qd=DNSQR(qname = domain))”

- IP(dst=DNS_SERVER): es el destino del paquete.
- /UDP(dport=53): protocolo DNS usa UDP puerto 53.
- DNS(...): es la capa de DNS, rd=1 es el “recursion desired” es querer la respuesta completa, “qd=” es el “query domain” y DNSQR(qname = domain) consulta el dominio generado.

Envío el paquete con “send(pkt, verbose=0”, envío el paquete a nivel 3 que es la (IP) y que no me genere logs.

Por último, imprimo cada consulta generada y le hago un time de 0,1 segundas entre consultas.

Ahora ejecuto el script y veo como funciona, pero teniendo en cuenta que tengo el detector activado también:

```
{(venv) } / # python3 dns_ataque.py
[*] Iniciando ataque de DNS snooping (simulado)...
[DNS QUERY] dghqsxnmoq.test.local
[DNS QUERY] ofdjyphwfh.test.local
[DNS QUERY] nbgekszolt.test.local
[DNS QUERY] rycjkpmyn.test.local
[DNS QUERY] lvillbxsue.test.local
[DNS QUERY] pgjezwrhuw.test.local
[DNS QUERY] iboipjfxgt.test.local
[DNS QUERY] djmrlidikwu.test.local
[DNS QUERY] bybwhpvpydn.test.local
[DNS QUERY] ktmfmupbzm.test.local
[DNS QUERY] eqxawyhtwn.test.local
[DNS QUERY] hyhimuehwt.test.local
[DNS QUERY] xvmbldjnhm.test.local
[DNS QUERY] tejkelbnix.test.local
[DNS QUERY] azxzkusemh.test.local
[DNS QUERY] spdrjiwmdt.test.local
[DNS QUERY] wpbtefwvcu.test.local
[DNS QUERY] ybhfcjwivp.test.local
[DNS QUERY] wpbrniveue.test.local
[DNS QUERY] sulmkduxmz.test.local
[DNS QUERY] mdxakqmdsh.test.local
[DNS QUERY] lotarlwlqi.test.local
[DNS QUERY] ctlegfhsqv.test.local
[DNS QUERY] qzodniwujd.test.local
[DNS QUERY] ndsxgdnppq.test.local
[DNS QUERY] mlvclwdly.test.local
[DNS QUERY] knhrzpjvkv.test.local
[DNS QUERY] xbnxkrhcnl.test.local
[DNS QUERY] bdnbujiouq.test.local
[DNS QUERY] bmjljiqgaq.test.local
[DNS QUERY] gdkqwwprxx.test.local
[DNS QUERY] axizaofmri.test.local
[DNS QUERY] xvptlqxtwi.test.local
[DNS QUERY] xkjahzdgin.test.local
[DNS QUERY] ozsxhprpm.test.local
[DNS QUERY] zzwzmayztn.test.local
[DNS QUERY] ovjagdkuue.test.local
```

Figura 32: Ejecución de script de ataque DNS

Ahora voy al resolver y veo sí funciona correctamente:

```
(venv) # python detector_dns.py
[*] Monitorizando consultas DNS...
[DNS] 10.0.0.10 → dghqsnmoq.test.local.
[DNS] 10.0.0.53 → dghqsnmoq.test.local.
[DNS] 10.0.0.10 → ofdjyphwfh.test.local.
[DNS] 10.0.0.53 → ofdjyphwfh.test.local.
[DNS] 10.0.0.10 → nbgekszolt.test.local.
[DNS] 10.0.0.53 → nbgekszolt.test.local.
[DNS] 10.0.0.10 → rycjkpmyn.test.local.
[DNS] 10.0.0.53 → rycjkpmyn.test.local.
[DNS] 10.0.0.10 → lvillbxsue.test.local.
[DNS] 10.0.0.53 → lvillbxsue.test.local.
[DNS] 10.0.0.10 → pgjezwrhuw.test.local.

[!!!] Posible DNS Snooping detectado
IP sospechosa: 10.0.0.10
Consultas DNS en 10 segundos: 6

[DNS] 10.0.0.53 → pgjezwrhuw.test.local.

[!!!] Posible DNS Snooping detectado
IP sospechosa: 10.0.0.53
Consultas DNS en 10 segundos: 6

[DNS] 10.0.0.10 → iboipjfxgt.test.local.

[!!!] Posible DNS Snooping detectado
IP sospechosa: 10.0.0.10
Consultas DNS en 10 segundos: 7

[DNS] 10.0.0.53 → iboipjfxgt.test.local.

[!!!] Posible DNS Snooping detectado
IP sospechosa: 10.0.0.53
Consultas DNS en 10 segundos: 7
```

Figura 33: Resultado detector DNS 1

```
[DNS] 10.0.0.10 → iboipjfxgt.test.local.

[!!!] Posible DNS Snooping detectado
IP sospechosa: 10.0.0.10
Consultas DNS en 10 segundos: 7

[DNS] 10.0.0.53 → iboipjfxgt.test.local.

[!!!] Posible DNS Snooping detectado
IP sospechosa: 10.0.0.53
Consultas DNS en 10 segundos: 7

[DNS] 10.0.0.10 → djmrldikwu.test.local.

[!!!] Posible DNS Snooping detectado
IP sospechosa: 10.0.0.10
Consultas DNS en 10 segundos: 8

[DNS] 10.0.0.53 → djmrldikwu.test.local.

[!!!] Posible DNS Snooping detectado
IP sospechosa: 10.0.0.53
Consultas DNS en 10 segundos: 8

[DNS] 10.0.0.10 → bybwhpvyn.test.local.

[!!!] Posible DNS Snooping detectado
IP sospechosa: 10.0.0.10
Consultas DNS en 10 segundos: 9

[DNS] 10.0.0.53 → bybwhpvyn.test.local.

[!!!] Posible DNS Snooping detectado
IP sospechosa: 10.0.0.53
Consultas DNS en 10 segundos: 9
```

Figura 34: Resultado detector DNS 2

El script detecta consultas DNS repetidas y masivas desde dos IPs:

10.0.0.10

10.0.0.53

Las marca como posible DNS Snooping detectado, en este sentido el DNS Snooping me lo detecta cuando un nodo genera muchas consultas DNS en poco tiempo.

En este sentido lo detecta porque registra cada consulta DNS, guarda cuántas hace, cada IP en una ventana de tiempo y si supera el umbral salta la alerta.

Por ejemplo consultas DNS en 10 segundos: 6-9, como supera el "threshold" que definí salta la alerta.

Las alertas de las dos IP no es que las dos me estén haciendo dicho ataque.

4. Resultados:

Tablas de Resultados — Detección de ARP Spoofing y DNS Snooping

1. Resultados de la detección de ARP Spoofing

Escenario utilizado

Elemento	IP	Función
app1	10.0.1.10	Víctima
router-1	10.0.1.254	Router / Gateway
attacker	10.0.1.50	Atacante

Descubrimiento de hosts con Bettercap

IP detectada	MAC detectada	Descripción
10.0.1.10	2a:f7:83:57:20:f0	Host víctima
10.0.1.254	c6:2a:68:93:87:8c	Router
10.0.1.50	7a:8c:35:4a:6b:a1	Atacante

Resultados observados durante la detección

Evento detectado	Resultado
Respuestas ARP repetidas	Detectadas
Tráfico ARP anómalo	Detectado
Flood ARP sospechoso	Detectado
Cambio de MAC para una misma IP	No observado
Alertas generadas por el detector	Sí

Salida observada del detector

Mensaje	Interpretación
ARP flood sospechoso: 10.0.1.50	El atacante genera respuestas ARP repetidas
ARP flood sospechoso: 10.0.1.254	El router participa en el intercambio ARP alterado
Monitorizando ARP...	El sistema de detección está activo

2. Resultados de la detección de DNS Snooping

Escenario utilizado

Elemento	IP	Función
client	10.0.0.10	Cliente / Atacante
resolver	10.0.0.53	DNS Resolver
dns-server	10.0.0.54	Servidor DNS

Configuración del detector DNS

Parámetro	Valor
THRESHOLD	5 consultas
WINDOW	10 segundos
Tipo de detección	Threshold Detection

Prueba básica con nslookup

Acción realizada	Resultado
nslookup google.com 10.0.0.53	El detector registró tráfico DNS
Consultas generadas automáticamente	google.com, com, net, gtld-servers
Alertas por volumen	Sí

Simulación de DNS Snooping con Scapy

Dominio generado	Resultado
dghqsnmoq.test.local	Consulta detectada
iboipjfxgt.test.local	Consulta detectada
djmrldikwu.test.local	Consulta detectada
bybwhpvyn.test.local	Consulta detectada

Resultados observados

Evento detectado	Resultado
Ráfaga de consultas DNS	Detectada
Consultas a subdominios inexistentes	Detectadas
Superación del threshold	Detectada
Alertas DNS Snooping	Generadas correctamente

Salida observada del detector DNS

Mensaje	Interpretación
[DNS] 10.0.0.10 -> dominio.test.local	Consulta DNS enviada por el cliente
[DNS] 10.0.0.53 -> dominio.test.local	El resolver procesa la consulta
Posible DNS Snooping detectado	Se superó el umbral de consultas
Consultas DNS en 10 segundos: 8	Activación de la detección por volumen

5. Conclusiones:

Gracias a esta práctica he podido comprender de forma práctica como funcionan algunos de los ataques más conocidos en las redes locales y sistemas DNS, así como la importancia de detectar comportamientos anómalos antes de que comprometan la seguridad de una infraestructura.

A través de la simulación de ataques de ARP Spoofing y DNS Snooping, he podido observar como un atacante puede manipular el tráfico de red o generar consultas DNS sospechosas para intentar obtener información o interceptar comunicaciones.

He aprendido sobre el papel fundamental de la monitorización y el análisis del tráfico en la ciberseguridad, mostrando pequeñas anomalías que pueden convertirse en indicadores clave de un posible ataque.

6. Bibliografía:

1. *Bettercap: framework para hacer MitM portable y modular.* (n.d.). Segu-Info - Ciberseguridad Desde 2000. <https://blog.segu-info.com.ar/2015/08/bettercap-framework-para-hacer-mitm.html?m=0>
2. *Envenenamiento de caché ARP | ESET Glossary.* (n.d.). Ayuda En Línea De ESET. https://help.eset.com/glossary/es-CL/arp_poisoning.html
3. Kaushika-Msft. (n.d.). *Ataques de snooping de caché del servidor DNS - Windows Server.* Microsoft Learn. <https://learn.microsoft.com/es-es/troubleshoot/windows-server/networking/dns-server-cache-snooping-attacks>